



JC4002 Knowledge Representation

Day 9: Web Ontology Language

Dr Ferdian Jovan

ferdian.jovan@abdn.ac.uk

Dr Aiden Durrant

aiden.durrant@abdn.ac.uk

Dr Jiangtian Nie

jiangtian.nie@abdn.ac.uk

Day 9

Web Ontology Language

1. OWL Background
2. OWL Syntax

Ontology

- ❑ The term ontology comes from **philosophy**: *it is the study of being*
- ❑ Philosophical ontologists ask, and try to answer, questions like “**what exists?**” which can lead to attempts at classifying everything; then “**what is matter?**”, “**what is time?**”, then further down to “**what is cost?**” (or, put differently, “**what is the nature of these things?**”)
- ❑ In **computer science**, ontologies are countable, and *an ontology* is a formal and explicit specification of the concepts, entities, and relationships that exist within a particular domain or knowledge base.
- ❑ A computer ontologists ask “what are the **classes** that we need to identify for this domain of knowledge? What are the **relations** that may exist between the **things** of interest in the domain? What are the **axioms** (equivalency, transitivity, reflexive, etc) that assert truth about certain kinds of things in the domain?”

Ontology

- ❑ Ontologies usually comprise:
 - A logical theory: formal axioms that are true of the domain
 - A documentation: definitions of what the terms in the theory are; what is their kind, e.g., a class vs. a relation; and what they are intended to describe (a natural language description)
- ❑ The language of ontologies are perfectly captured by Description Logic, in particular $\mathcal{AL}(\mathcal{C})$ logic.
- ❑ However, $\mathcal{AL}$ logic does not cover all formal axioms necessary to describe what are true in the world. Some extensions are needed to accompany $\mathcal{AL}$ logic.
- ❑ There are many variations of $\mathcal{AL}$ logic in combinations with various axioms, e.g. $\mathcal{ALHI}$ denotes the $\mathcal{AL}$ that has role hierarchy and role inverse as its extra axioms.
- ❑ This time we will focus on the extensions that make up OWL.

ALC Extensions for OWL: Inverse Roles(*I*)

- ❑ Assume that we have ABox **A**:

Adult(JohnSmith) **AffectedBy**(JohnSmith, JRA) **Affects**(JohnSmith, JRA)

- ❑ We would believe that ABox **A** would lead to an inconsistency.
- ❑ However, this is not the case, because there is no semantic relationship between **Affects** and **AffectedBy**.
- ❑ In order to relate roles such as **Affects** and **Affects** in the desired way, DLs can be extended with inverse roles.
- ❑ The fact that a DL provides inverse roles is normally indicated by the letter *I* in its name, e.g., *ALCI*.

$\mathcal{ALC}$ Extensions for OWL: Inverse Roles($\mathcal{I}$)

- ❑ Let $\mathbf{R}$ be the set of **role** names. For $R \in \mathbf{R}$, R^- is an *inverse role*. The set of $\mathcal{I}$ roles is $\mathbf{R} \cup \{R^- \mid R \in \mathbf{R}\}$.
- ❑ Let $\mathcal{ALC}$ be a description logic. The set of $\mathcal{ALCI}$ **concepts** is the smallest set of **concepts** that contains all $\mathcal{ALC}$ **concepts** and where $\mathcal{I}$ **roles** can occur in all places of **role** names.
- ❑ An interpretation $\mathcal{I}$ maps inverse roles to binary relations as follows:

$$\varphi(R) = \{(y, x) \mid (x, y) \in \varphi(R)\}$$

- ❑ Typically, DLs supporting inverse roles also allow for inverse roles to be used in axioms such as the following:

$$\text{AffectedBy} \doteq \text{Affects}^-$$

which establishes the intuitive semantic relationship.

ALC Extensions for OWL: Number Restriction(*Q*)

- ❑ We might want to state that **MildArthritis** **Affects** at most 2 **Joints**, or that **SevereArthritis** **Affects** at least 5 **Joints**.
- ❑ In order to support this, DLs can be extended with (qualified) number restrictions, usually indicated by $\mathcal{N}$ for NRs and $\mathcal{Q}$ for QNRs.
- ❑ NRs are concept descriptions whose instances are related to at least/most n other individuals via a given role; e.g., (≤ 2 **sister**) describes individuals having at most 2 sisters.
- ❑ It is equivalent to the old DL [EXISTS 2 **:Sister**]
- ❑ QNRs additionally allow for restricting the type of the target individuals; e.g., (≥ 2 **sister.Graduate**) describes individuals having at least 2 sisters who are graduates.
- ❑ Note that an NR is equivalent to a QNR where the restriction concept is **T**; e.g., (≤ 2 **sister**) is equivalent to (≤ 2 **sister.T**).

ALC Extensions for OWL: Number Restriction(*Q*)

- For n a non-negative number, r an *ALC* role and C a (possibly complex) *ALC* concept description, a *number restriction* is a concept description of the form $(\leq n r)$ or $(\geq n r)$, and a *qualified number restriction* is a concept description of the form $(\leq n r.C)$ or $(\geq n r.C)$.
- For an interpretation $\zeta = (\delta, \varphi)$, its mapping φ is extended as follows, where $\#M$ is used to denote the cardinality of a set M :
 - $\varphi(\leq n r) = \{d \in \delta \mid \#\{e \mid (d, e) \in \varphi(r)\} \leq n\},$
 - $\varphi(\geq n r) = \{d \in \delta \mid \#\{e \mid (d, e) \in \varphi(r)\} \geq n\},$
 - $\varphi(\leq n r.C) = \{d \in \delta \mid \#\{e \mid (d, e) \in \varphi(r) \text{ and } e \in \varphi(C)\} \leq n\},$
 - $\varphi(\geq n r.C) = \{d \in \delta \mid \#\{e \mid (d, e) \in \varphi(r) \text{ and } e \in \varphi(C)\} \geq n\}.$
- Concept descriptions $(=n r)$ and $(=n r.C)$ may be used as abbreviations for $(\leq n r) \sqcap (\geq n r)$ and $(\leq n r.C) \sqcap (\geq n r.C)$, respectively.

ALC Extensions for OWL: Nominal($\mathcal{O}$)

- ❑ So far our use of individuals has been restricted to ABox axioms.
- ❑ We may also want to use individuals in concept descriptions; e.g., to describe those individuals who are affected by some **Disease** that also affects the individual **JohnSmith**.
- ❑ Intuitively, we might try the description

$$\exists \text{Affects}^-(\text{Disease} \sqcap \exists \text{Affects}.\text{JohnSmith})$$

But this will not work, because in this context **JohnSmith** must be a **concept**.

- ❑ Nominals allow for the construction of a concept from an individual name; e.g.: $\{\text{JohnSmith}\}$ is the concept whose only instance is **JohnSmith**.
- ❑ The fact that a DL provides nominals is normally indicated by the letter $\mathcal{O}$ in its name ($\mathcal{N}$ is already used for unqualified number restrictions).

$\mathcal{ALC}$ Extensions for OWL: Nominal($\mathcal{O}$)

- ❑ Let $\mathbf{U}$ be the set of individual names. For $b \in \mathbf{U}$, $\{b\}$ is called a *nominal*.
- ❑ Let $\mathcal{ALC}$ be a description logic. The description logic $\mathcal{ALCO}$ is obtained from $\mathcal{ALC}$ by allowing nominals as additional concepts.
- ❑ For an interpretation $\zeta = (\delta, \varphi)$, its mapping φ is extended as follows:

$$\varphi(\{b\}) = \{\varphi(b)\}$$

- ❑ We can now form the desired concept description:

$$\exists \text{Affects}^-. (\text{Disease} \sqcap \exists \text{Affects}. \{\text{JohnSmith}\})$$

- ❑ With nominals, the separation between ABox and TBox is not meaningful:

$$C(a) \equiv \{a\} \sqsubseteq C$$

$$R(a, b) \equiv \{a\} \sqsubseteq \exists R. \{b\}$$

$\mathcal{ALC}$ Extensions for OWL: Role Hierarchies ($\mathcal{H}$)

- ❑ We may want our KB to provide some structure for roles as well as concepts; e.g.: we may want to state that roles **brother** and **sister** are subsumed by the role **sibling**.
- ❑ The fact that a DL provides such role inclusion axioms (RIAs) is normally indicated by the letter $\mathcal{H}$ in its name (there is a Hierarchy of roles).
- ❑ Definition:
 - A *role inclusion axiom* (RIA) is an axiom of the form $r \sqsubseteq s$ for r, s $\mathcal{ALC}$ roles.
 - The DL $\mathcal{ALCH}$ is obtained from $\mathcal{ALC}$ by allowing, additionally, role inclusion axioms in TBoxes.
 - For an interpretation $\zeta = (\delta, \varphi)$ to be a model of a role inclusion axiom $r \sqsubseteq s$, it has to satisfy $\varphi(r) \subseteq \varphi(s)$

ALC Extensions for OWL: Transitive Roles

- We can use the role `parent` to form descriptions such as:

$\exists \text{parent}.\text{Irish}$ having an Irish parent

$\exists \text{parent} . (\exists \text{parent} . \text{Irish})$ having an Irish grandparent

$\exists \text{parent} . (\exists \text{parent} . (\exists \text{parent} . \text{Irish}))$ having an Irish greatgrandparent

- But what if we want to mention Irish ancestors without specifying a generation?

- We can do that by using a combination of role hierarchy and transitive roles:

$\text{parent} \sqsubseteq \text{ancestor}$

`parent` is a sub-role of `ancestor`

$\text{Trans}(\text{ancestor})$

`ancestor` is a transitive role

$\exists \text{ancestor} . \text{Irish}$

having an Irish ancestor

ALC Extensions for OWL: Transitive Roles

- ❑ A *role transitivity axiom* is an axiom of the form $\text{Trans}(r)$ for r an *ALC* role.
- ❑ The name of the DL that is the extension of *ALC* by allowing, additionally, transitivity axioms in TBoxes, is usually given by replacing *ALC* in $\mathcal{L}$'s name with $\mathcal{S}$.
- ❑ For an interpretation $\zeta = (\delta, \varphi)$ to be a model of a role transitivity axiom $\text{Trans}(r)$, the relation $\varphi(r)$ must be **transitive**.
- ❑ If the role r is defined as transitive (i.e. $\text{Trans}(r)$), then any relationship extended over multiple steps of this role implies a direct relationship under the **same role r** .
- ❑ The use of $\mathcal{S}$ to replace *ALC* in DLs with transitive roles is inspired by similarities with the modal logic **S4** (and a desire for shorter names).
- ❑ However, in some cases (the subscript) $\cdot R^+$ is used to indicate transitive roles; e.g., *SHIQ* could be written *ALCHIQ*_{R⁺}.

Transitive Roles VS Transitive Closure

- ❑ It is important to understand the difference between *transitive roles* and the *transitive closure* of roles.
- ❑ Transitive closure is a role constructor: given a role r , transitive closure can be used to construct a role r^+ , with the semantics being that $\varphi(r^+) = (\varphi(r))^+$.
- ❑ Transitive closure, indirectly, creates a new role r^+ that includes not just direct relationships but also all relationships implied by repeated applications of the original role.
- ❑ The minimal description logic that can simulate transitive closure of roles is $\mathcal{SH}$. Adding axioms $\text{Trans}(s)$ and $r \sqsubseteq s$ to a TBox T ensures that in every model ζ of T , $\varphi(s)$ is transitive, and $\varphi(r) \subseteq \varphi(s)$.
- ❑ However, $\mathcal{SH}$ does not explicitly allow you to directly express the transitive closure operation as a separate construct r^+ .

Transitive Roles VS Transitive Closure

- ❑ With $\mathcal{S}$ (transitive roles), we can define:
 - ❑ $T = \{\text{Trans}(\text{parent}), \text{ancestor} \sqsubseteq T\}$
 - ❑ $A = \{\text{parent}(\text{john}, \text{mary}), \text{parent}(\text{mary}, \text{sue})\}$
 - ❑ $\{T, A\} \models \text{parent}(\text{john}, \text{sue})$
 - ❑ But, $\{T, A\} \not\models \text{ancestor}(\text{john}, \text{sue})$ and $\{T, A\} \not\models \text{parent}^+(\text{john}, \text{sue})$
- ❑ With $\mathcal{SH}$ (transitive roles + hierarchy roles), we can define:
 - ❑ $T = \{\text{Trans}(\text{ancestor}), \text{parent} \sqsubseteq \text{ancestor}\}$
 - ❑ $A = \{\text{parent}(\text{john}, \text{mary}), \text{parent}(\text{mary}, \text{sue})\}$
 - ❑ $\{T, A\} \models \text{parent}(\text{john}, \text{sue}) \sqsubseteq \text{ancestor}(\text{john}, \text{sue})$
 - ❑ But, $\{T, A\} \not\models \text{parent}^+(\text{john}, \text{sue})$
 - ❑ $\mathcal{SH}$ indirectly simulates transitive closure behaviour (e.g. ancestor simulates parent^+)
 - ❑ If transitive closure were added:
 - ❑ $\{T, A\} \models \text{parent}(\text{john}, \text{sue}) \sqsubseteq \text{parent}^+(\text{john}, \text{sue}) \sqsubseteq \text{ancestor}(\text{john}, \text{sue})$

OWL (Web Ontology Language)

OWL is an ontology language standard for web applications of ontologies (the semantic web). OWL is used in “web-independent” applications as well.

- ❑ OWL 1.0 language is a World Wide Web Consortium (W3C) recommendation since February 2004.
 - ***SHIQ(D)*** is the basis of OWL 1.0
 - Note that ***S*** is used as a shortcut for ***ALC*** + transitivity
 - ***(D)*** refers to the support for **datatypes**. It allows the incorporation of concrete domains such as integers, strings, or dates, which can be used in property restriction.
- ❑ Many applications (for example medicine) have requirements to specify interactions between roles (that is **not** in the OWL 1.0 specification):
 - ❑ E.G.: A fracture located in part of the Femur is a fracture of the Femur
 - ❑ This property chains **can't** be expressed in OWL 1.0

OWL (Web Ontology Language)

OWL is an ontology language standard for web applications of ontologies (the semantic web). OWL is used in “web-independent” applications as well.

□ OWL 2.0 language is a new version, it is a W3C recommendation since October 2009

- ***SRIOQ(D)*** is the basis of OWL 2.0
- ***R*** covers the role of ***H***ierarchy as well as the **role chain**
- **(D)** covers richer datatypes such as user defined datatypes
- It also includes additional expressivity:
 - Qualified cardinality restrictions
 - Local reflexivity restrictions
 - Reflexive/Irreflexive/Symmetric/Asymmetric properties
 - Disjoint properties.

OWL (Web Ontology Language)

❑ OWL 2 ontologies provide

- classes,
- properties,
- individuals, and
- data values

and are stored as Semantic Web documents. OWL 2 ontologies can be used along with information written in RDF (Resource Description Framework), and OWL 2 ontologies themselves are primarily exchanged as RDF documents.

❑ OWL 2 has five languages: OWL2 Full, OWL2 DL, and the profiles OWL2 QL, OWL2 EL, and OWL2 RL.

OWL2 Full

- ❑ It is very expressive language;
- ❑ All OWL features added on top of RDF(S)
 - Allows, e.g., redefining the meaning of RDF(S) and OWL primitives
- ❑ It is fully upward-compatible with RDF (syntactically and semantically):
 - any legal RDF document is also a legal OWL Full document
 - any valid RDF/S conclusion is also a valid OWL Full conclusion
- ❑ RDF-based semantics
- ❑ However, OWL2 Full is **undecidable**
- ❑ Complete and efficient reasoning not possible

OWL2 DL (Description Logic)

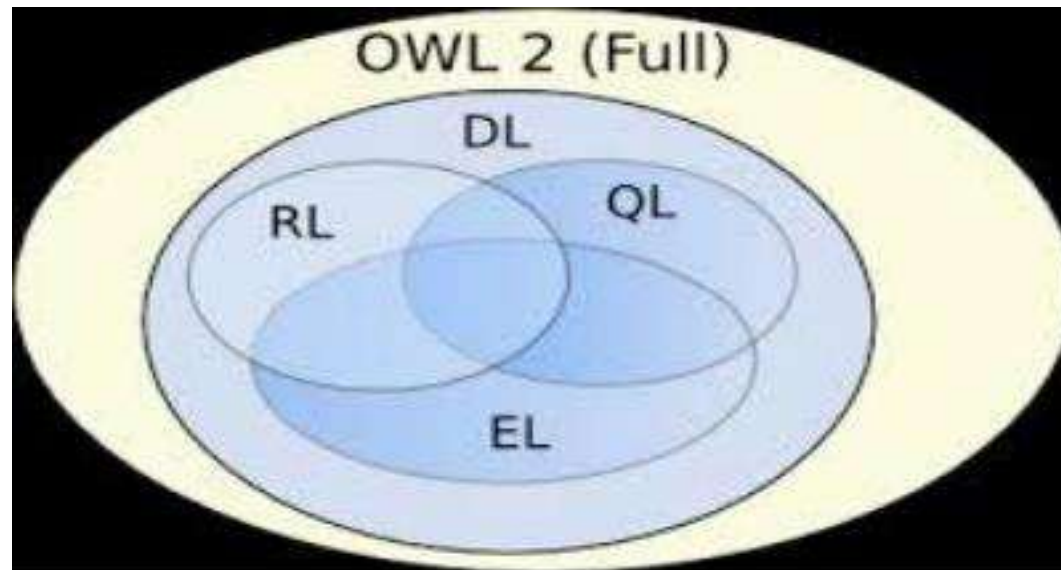
- ❑ Restricted form of OWL Full for which decidable, efficient support for reasoning is possible
 - OWL 2 primitives cannot be applied to themselves
 - Only classes of non-literal resources considered
 - Strict separation between datatype and object property
 - Strict separation between an individual, a class, or a property
- ❑ It corresponds approximately to *SR10Q* with XML datatypes.
- ❑ Direct semantics, based on Description logics (Terminology logics)
 - Subsets of predicate logic
 - But also RDF-based semantics can be applied to OWL2 DL ontologies
- ❑ However OWL2 DL is still not **tractable**.

OWL 2 Profiles

OWL 2 DL includes three specific profiles for different use cases

- ❑ **OWL 2 EL** ($\mathcal{EL}$ Description Logic)
- ❑ **OWL 2 QL** (DL-Lite Query Language)
- ❑ **OWL 2 RL** (Rule Language)

Each profile includes a subset of OWL DL features



OWL 2 EL ($\mathcal{EL}$ Description Logic)

- ❑ It is based on the description logic $\mathcal{EL}$.
- ❑ Good for ontologies with lots of classes and/or properties
- ❑ Polynomial complexity of standard inference types: satisfiability, classification, instance checking
- ❑ Used for large scale class ontologies, e.g., Snomed CT
- ❑ Its expressive power is traded for performance guarantees.
- ❑ Limitations include:
 - Negation and disjunction not supported
 - Universal quantification on properties like “all children of a rich person are rich” can’t be expressed.
 - All kinds of role inverses are not available
 - E.g., **parentOf** and **childOf** can’t be stated as inverses

OWL 2 QL (DL-Lite Query Language)

- ❑ It is based on description logics similar to DL-Lite.
- ❑ It enables conjunctive queries to be answered in LogSpace (more precisely, $\mathcal{AC}_0$) using standard relational database technology;
- ❑ It is particularly suitable for applications where relatively lightweight ontologies are used to organize large numbers of individuals and where it is useful or necessary to access the data directly via relational queries (e.g., SQL).
- ❑ Limitations include:
 - Existential quantification of roles to a class expression
 - E.g., *it can be stated that every person has a parent, but not that every person has a female parent*
 - Property chain axioms and equality are not supported

OWL 2 RL (Rule Language)

- ❑ **OWL 2 RL** enables the implementation of polynomial time reasoning algorithms using rule-extended database technologies operating directly on RDF triples;
- ❑ Good for rule-based reasoning, database focus
- ❑ Can be implemented using logic programming
 - First-order implications: IF certain triples exist THEN add additional triples
- ❑ Limitations include:
 - Statements where the existence of an individual enforces the existence of another individual
 - E.g., the statement “every person has a parent” is not expressible
 - Restricts class axioms asymmetrically
 - Constructs for a subclass cannot necessarily be used as a superclass

Logical Assumptions of OWL

- ❑ The standard semantics of OWL makes an **Open World Assumption** (OWA).
 - ❑ We cannot assume that all information is known about all the individuals in a domain.
 - ❑ Example: If an ontology does not explicitly state that an individual **A** is related to an individual **B** through a certain **property**, OWL does not assume that **A** is not related to **B**; it merely means that the ontology lacks information about this relationship.
 - ❑ However, in some extreme cases, lots of additional assertions may be needed for 'closing' data.
- ❑ Prolog, on the other hand, is based on Closed World Assumption (CWA):
what is not known to be true is false
 - ❑ CWA assumes that everything is known, and data not stated is assumed to be false;

Logical Assumptions of OWL

- ❑ Typical database applications assume that individuals with same/different names are indeed same/different individuals
- ❑ OWL follows the usual logical paradigm where this is **not** the case (**Non-Unique Names Assumption -- NUNA**).
 - ❑ two distinct identifiers might actually refer to the same real-world entity, unless explicitly stated otherwise.
 - ❑ Example: If we have two individuals ex:John and ex:Johnny, OWL does not automatically assume they are different individuals. To assert that they are different, an explicit statement using 'owl:differentFrom' is required.
- ❑ The ontologists may want to indicate portions of the ontology for which the assumption does or does not hold

Day 9

Web Ontology Language

1. OWL Background
- 2. OWL Syntax**

Main Components of an OWL Ontology

□ Classes

- Class definitions = “constructors”

Concepts (e.g. Bird, Eagle)

□ Properties

- Object properties
- Datatype properties
- Annotation properties

Classes are defined using properties

(to define roles, e.g., married)

(to link individuals to the domain δ data values such as string, numbers, dates)

(ontology explanation for human users)

□ Individuals

(e.g. bob)

The OWL Language

There are different syntactic forms of OWL:

- ❑ RDF' XML-based syntax (primary syntax for OWL)
- ❑ Specific OWL/XML schema that does not follow the RDF conventions
 - more easily read by human users
 - see <http://www.w3.org/TR/owl-xmlsyntax/>
- ❑ **A functional-style syntax**
 - used in the language specification document
 - much more compact and readable
 - see <http://www.w3.org/TR/owl-semantic/>
- ❑ Turtle (Terse RDF Triple Language) Syntax
 - ❑ more human-readable as it uses a simplified, concise syntax.

XML/RDF-based: OWL ontologies (header)

```
<rdf:RDF xmlns:owl="http://www.w3.org/2002/07/owl#"
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
  xml:base="http://www.dcs.bbk.ac.uk/">

  <owl:Ontology rdf:about="">
    <rdfs:comment>An example OWL ontology</rdfs:comment>
    <owl:priorVersion rdf:resource="http://www.dcs.bbk.ac.uk/uni-old-ns"/>
    <owl:imports rdf:resource="http://www.dcs.bbk.ac.uk/person"/>
    <rdfs:label>SCSIS Ontology</rdfs:label>
  </owl:Ontology>

  ...

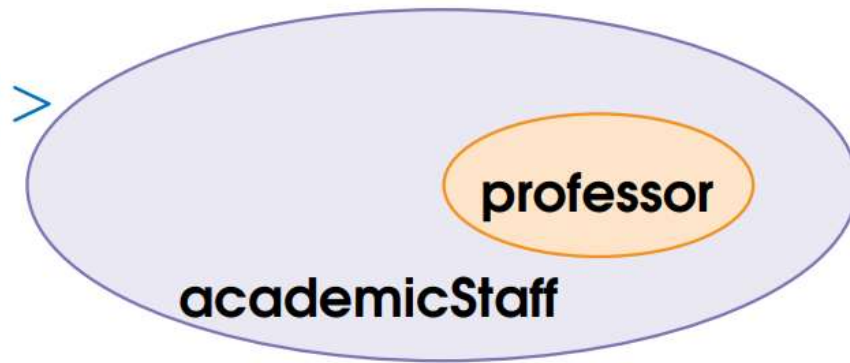
</rdf:RDF>
```

XML/RDF-based: The OWL language (classes)

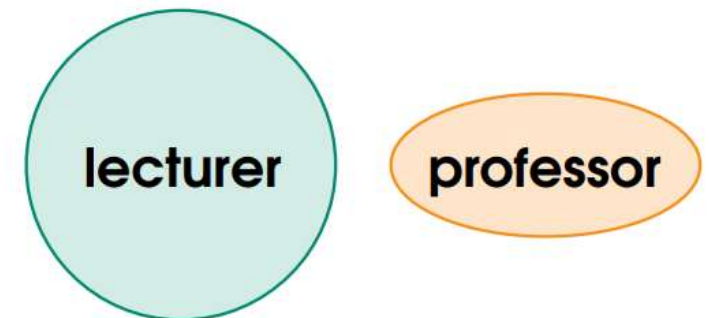
Classes are defined using an **owl:Class** element

(**owl:Class** is a subclass of **rdfs:Class**)

```
<owl:Class rdf:ID="professor">  
  <rdfs:subClassOf rdf:resource="#academicStaff" />  
</owl:Class>
```



```
<owl:Class rdf:about="#professor">  
  <owl:disjointWith rdf:resource="#lecturer" />  
</owl:Class>
```



Examples on the Next Slides are Taken from This [OWL 2 Primer](#)

W3C Recommendation



OWL 2 Web Ontology Language Primer (Second Edition)

W3C Recommendation 11 December 2012

This version:
<http://www.w3.org/TR/2012/REC-owl2-primer-20121211/>

Latest version (series 2):
<http://www.w3.org/TR/owl2-primer/>

Latest Recommendation:
<http://www.w3.org/TR/owl-primer>

Previous version:
<http://www.w3.org/TR/2012/PER-owl2-primer-20121018/>

Editors:
[Pascal Hitzler](#), Wright State University
[Markus Krotzsch](#), University of Oxford
[Bijan Parsia](#), University of Manchester
Peter F. Patel-Schneider, Nuanace Communications
[Sebastian Rudolph](#), FZI Research Center for Information Technology

Please refer to the [errata](#) for this document, which may include some normative corrections.

A [color-coded version of this document showing changes made since the previous version](#) is also available.

This document is also available in these non-normative formats: [PDF version](#).

See also [translations](#).

Copyright © 2012 W3C® ([MIT](#), [ERCIM](#), [Keio](#)). All Rights Reserved. W3C [liability](#), [trademark](#) and [document use](#) rules apply.

Abstract

The OWL 2 Web Ontology Language, informally OWL 2, is an ontology language for the Semantic Web with formally defined meaning. OWL 2 ontologies provide classes, properties, individuals, and data values and are stored as Semantic Web documents. OWL 2 ontologies can be used along with information written in RDF, and OWL 2 ontologies themselves are primarily exchanged as RDF documents. The OWL 2 [Document Overview](#) describes the overall state of OWL 2, and should be read before other OWL 2 documents.

This primer provides an approachable introduction to OWL 2, including orientation for those coming from other disciplines, a running example showing how OWL 2 can be used to represent first simple information and then more complex information, how OWL 2 manages ontologies, and finally the distinctions between the various sublanguages of OWL 2.

Classes, hierarchies, individuals for the functional syntax

- ☐ Classes and Individuals
- ☐ Subclass relations
- ☐ Class equivalence and disjointness
- ☐ Individual equivalence and disjointness

Functional-Style Syntax

```
ClassAssertion( :Woman :Mary )
```

Functional-Style Syntax

```
SubClassOf( :Woman :Person )
```

Functional-Style Syntax

```
EquivalentClasses( :Person :Human )
```

Functional-Style Syntax

```
DisjointClasses( :Woman :Man )
```

Functional-Style Syntax

```
SameIndividual( :James :Jim )
```

Functional-Style Syntax

```
DifferentIndividuals( :John :Bill )
```

Classes, hierarchies, individuals for the functional syntax

❑ Enumerations

Defining a class by its members

```
Functional-Style Syntax
EquivalentClasses(
  :MyBirthdayGuests
  ObjectOneOf( :Bill :John :Mary)
)
```

❑ Complex classes

○ Union

```
Functional-Style Syntax
EquivalentClasses(
  :Parent
  ObjectUnionOf( :Mother :Father )
)
```

○ Intersection

```
Functional-Style Syntax
EquivalentClasses(
  :Mother
  ObjectIntersectionOf( :Woman :Parent )
)
```

○ Complement

*Jack is a person
but not a parent*

```
Functional-Style Syntax
ClassAssertion(
  ObjectIntersectionOf(
    :Person
    ObjectComplementOf( :Parent )
  )
  :Jack
)
```

Quantification for the functional syntax

Class constructors based on quantified property values

☐ Universal restrictions

*All children of a happy person are happy
(may have no children, too)*

Functional-Style Syntax

```
EquivalentClasses(  
  :HappyPerson  
  ObjectAllValuesFrom( :hasChild :HappyPerson )  
)
```

☐ Existential restrictions

A parent has at least one child

Functional-Style Syntax

```
EquivalentClasses(  
  :Parent  
  ObjectSomeValuesFrom( :hasChild :Person )  
)
```

☐ Using several conditions

Functional-Style Syntax

```
EquivalentClasses(  
  :HappyPerson  
  ObjectIntersectionOf(  
    ObjectAllValuesFrom( :hasChild :HappyPerson )  
    ObjectSomeValuesFrom( :hasChild :HappyPerson )  
  )  
)
```

Property Cardinality Restrictions for the functional syntax

- ☐ **Max cardinality**

John has at most 4 children

Functional-Style Syntax

```
ClassAssertion(  
  ObjectMaxCardinality( 4 :hasChild :Parent )  
  :John  
)
```

- ☐ **Min cardinality**

John has at least 2 children

Functional-Style Syntax

```
ClassAssertion(  
  ObjectMinCardinality( 2 :hasChild :Parent )  
  :John  
)
```

- ☐ **Exact cardinality**

John has 3 children

Functional-Style Syntax

```
ClassAssertion(  
  ObjectExactCardinality( 3 :hasChild :Parent )  
  :John  
)
```

More Property Restrictions for the functional syntax

❑ **Value** restrictions

Functional-Style Syntax

```
EquivalentClasses(  
  :JohnsChildren  
  ObjectHasValue( :hasParent :John )  
)
```

❑ **Self** restrictions

Functional-Style Syntax

```
EquivalentClasses(  
  :NarcisticPerson  
  ObjectHasSelf( :loves )  
)
```

Class and Property Types

❑ **Class** relates to **concept** in DL

Each class must be declared, e.g., `Declaration(Class(:Cat))`

❑ **Annotation** relates to comments aimed at human users

- Each annotation must be declared, e.g.,
`Declaration(AnnotationProperty(terms:contributor))`
- Once declared it can be used, e.g.,
`Annotation(terms:contributor "Ferdian Jovan")`
- Some annotations are built-in (no need to declare anything), e.g.,
`Annotation(rdfs:label "hogwarts"),`
`Annotation(rdfs:comment "It's all about Harry Potter")`

Class and Property Types

- ❑ **Object properties** relate to **roles** in DL

Each object property must be declared, e.g.,

Declaration(ObjectProperty(:hasPet))

- ❑ **Named Individuals** relate to **individuals** in DL

Each individual must be declared, e.g.,

Declaration(NamedIndividual(:Dumbledore))

- ❑ **Datatype properties** relate individuals to literals of certain datatypes

- Each datatype property must be declared, e.g.,

Declaration(DataProperty(:age))

Property Characteristic for the functional syntax

☐ Inverse properties

Functional-Style Syntax

```
InverseObjectProperties( :hasParent :hasChild )
```

☐ Symmetric and
Asymmetric

Functional-Style Syntax

```
SymmetricObjectProperty( :hasSpouse )
```

Functional-Style Syntax

```
AsymmetricObjectProperty( :hasChild )
```

☐ Disjointness

Functional-Style Syntax

```
DisjointObjectProperties( :hasParent :hasSpouse )
```

☐ Reflexive and
Irreflexive

Functional-Style Syntax

```
ReflexiveObjectProperty( :hasRelative )
```

Functional-Style Syntax

```
IrreflexiveObjectProperty( :parentOf )
```


Property Characteristic for the functional syntax

❑ **Transitive** properties

Functional-Style Syntax

```
TransitiveObjectProperty( :hasAncestor )
```

❑ **Functional** properties

$$T \sqsubseteq \leq 1 \text{ } r.T$$

Functional-Style Syntax

```
FunctionalObjectProperty( :hasHusband )
```

❑ **Inverse-functional** properties

$$T \sqsubseteq \leq 1 \text{ } r.T$$

Functional-Style Syntax

```
InverseFunctionalObjectProperty( :hasHusband )
```

❑ **Subproperty** relations and
Property chains

Functional-Style Syntax

```
SubObjectPropertyOf(  
  ObjectPropertyChain( :hasParent :hasParent )  
  :hasGrandparent  
)
```

❑ **Keys:** Identify uniquely
individuals by values of key
properties

```
HasKey( :RegisteredPatient :hasWaitingListN )
```

```
ClassAssertion
```

```
( :RegisteredPatient :ThisPatient )
```

```
DataPropertyAssertion
```

```
( :hasWaitingListN :ThisPatient "123-45-6789" )
```

Setting property values for the functional syntax

❑ Object property values

Functional-Style Syntax

```
ObjectPropertyAssertion( :hasWife :John :Mary )
```

Functional-Style Syntax

```
NegativeObjectPropertyAssertion( :hasWife :Bill :Mary )
```

❑ Domain / Range restrictions

Domain(**C**) $T \sqsubseteq \forall r.C$

Range(**C**) $\exists r.T \sqsubseteq C$

Functional-Style Syntax

```
ObjectPropertyDomain( :hasWife :Man )  
ObjectPropertyRange( :hasWife :Woman )
```

❑ Datatype property

Functional-Style Syntax

```
DataPropertyAssertion( :hasAge :John "51"^^xsd:integer )
```

Functional-Style Syntax

```
DataPropertyDomain( :hasAge :Person )  
DataPropertyRange( :hasAge xsd:nonNegativeInteger )
```

Examples of Various OWL Syntaxes

Today we just focus on one of these syntaxes (Functional-Style Syntax)

But

You can see and learn different syntaxes on the Primer!

Functional-Style Syntax

```
ClassAssertion( :Person :Mary )
```

RDF/XML Syntax

```
<Person rdf:about="Mary"/>
```

Turtle Syntax

```
:Mary rdf:type :Person .
```

Manchester Syntax

```
Individual: Mary  
Types: Person
```

OWL/XML Syntax

```
<ClassAssertion>  
  <Class IRI="Person"/>  
  <NamedIndividual IRI="Mary"/>  
</ClassAssertion>
```

Examples of Various OWL Syntaxes

Functional-Style Syntax

```
EquivalentClasses(  
  :Mother  
  ObjectIntersectionOf( :Woman :Parent )  
)
```

RDF/XML Syntax

```
<owl:Class rdf:about="Mother">  
  <owl:equivalentClass>  
    <owl:Class>  
      <owl:intersectionOf rdf:parseType="Collection">  
        <owl:Class rdf:about="Woman"/>  
        <owl:Class rdf:about="Parent"/>  
      </owl:intersectionOf>  
    </owl:Class>  
  </owl:equivalentClass>  
</owl:Class>
```

Turtle Syntax

```
:Mother owl:equivalentClass [  
  rdf:type owl:Class ;  
  owl:intersectionOf ( :Woman :Parent )  
] .
```

Manchester Syntax

```
Class: Mother  
EquivalentTo: Woman and Parent
```

OWL/XML Syntax

```
<EquivalentClasses>  
  <Class IRI="Mother"/>  
  <ObjectIntersectionOf>  
    <Class IRI="Woman"/>  
    <Class IRI="Parent"/>  
  </ObjectIntersectionOf>  
</EquivalentClasses>
```

Example OWL ontology

We want to define:

- ❑ **HotWeather** class as a **WeatherObservation** with temperature ≥ 30 degrees Celsius
- ❑ **HumidWeather** is a subclass of **HotWeather** with precipitation < 0.1
- ❑ An **observation** of **WeatherObservation** with temperature = 32.5, and precipitation = 0.09

in one OWL ontology.

```
# The standard prefix needed for functional-syntax (.ofn)
Prefix(:=<http://example.org#>)
Prefix(rdfs:=<http://www.w3.org/2000/01/rdf-schema#>)
Prefix(owl:=<http://www.w3.org/2002/07/owl#>)
Prefix(xsd:=<http://www.w3.org/2001/XMLSchema#>)
```

Ontology(

Data Declarations

Declaration(Class(:HotWeather))

Declaration(Class(:HumidWeather))

Declaration(Class(:WeatherObservation))

Declaration(DataProperty(:temperature))

Declaration(DataProperty(:precipitation))

Data Properties Definitions

DataPropertyDomain(:temperature :WeatherObservation)

DataPropertyRange(:temperature xsd:decimal)

DataPropertyDomain(:precipitation :WeatherObservation)

DataPropertyRange(:precipitation xsd:decimal)

Example OWL ontology

CONTINUATION FROM PREVIOUS CODE PART 2

Class Definitions

EquivalentClasses(:HotWeather

ObjectIntersectionOf(

:WeatherObservation

DataAllValuesFrom(

:temperature

DatatypeRestriction(

xsd:decimal

xsd:minInclusive "30"^^xsd:decimal))))

CODE PART 3

Individuals and Assertions

Declaration(NamedIndividual(:Observation1))

ClassAssertion(:WeatherObservation :Observation1)

DataPropertyAssertion(:temperature :Observation1 "32.5"^^xsd:decimal)

DataPropertyAssertion(:precipitation :Observation1 "0.09"^^xsd:decimal)

)

SubClassOf(:HumidWeather

ObjectIntersectionOf(

:HotWeather

DataAllValuesFrom(

:precipitation

DatatypeRestriction(

xsd:decimal

xsd:maxExclusive "0.1"^^xsd:decimal))))

Example OWL ontology

CODE PART 3

Individuals and Assertions

Declaration(NamedIndividual(:Observation1))

ClassAssertion(:WeatherObservation :Observation1)

DataPropertyAssertion(:temperature :Observation1 "32.5"^^xsd:decimal)

DataPropertyAssertion(:precipitation :Observation1 "0.09"^^xsd:decimal)

) # Closing bracket of the Ontology

Graph Ontology representation (produced by Protégé)



Summary

- OWL 2.0 is based on *SRIQ(D)* Description Logic
- OWL 2.0 extends RDF(S)
- Multiple corresponding syntaxes:
 - RDF notations (.rdf, Turtle, etc.), OWL / XML, Functional-style
- Two OWL versions
 - OWL 2 DL: Direct semantics based on Description Logics
 - Decidable, efficient reasoning
 - Not fully upward compatible with RDF(S)
 - OWL 2 Full: Based on RDF semantics
 - Undecidable, partial reasoning possible
 - Upward compatible with RDF(S)